

Imágenes HDR (High Dynamic Range)

1) Lectura de las imágenes y extracción de datos a usar

Las imágenes `belg_1.jpg`, `belg_2.jpg`, ... son un "dataset" usado habitualmente para probar los algoritmos de HDR. Corresponden a $P=9$ exposiciones en color con tiempos de $1/1000$, $1/500$, $1/250$, $1/125$, $1/60$, $1/30$, $1/15$, $1/8$ y $1/4$ (en segundos). Cread un vector `T` con los tiempos de las P exposiciones.

El algoritmo usado se basa en determinar los valores de la función $g(Z)=\log_2(E)$ que relaciona la cantidad de luz recibida en un punto del sensor E con el valor que toma ese píxel en la foto. Como la función $g(Z)$ solo nos interesa en los valores de los píxeles $(0,1,\dots, 255)$ basta conocerla en dichas posiciones $(g_0, g_1, \dots, g_{255})$. Notad que debido al indexado de Matlab g_0 se guarda en $g(1)$, g_1 en $g(2)$, ... En principio esta función $g()$ sería distinta para cada plano de color, pero en la práctica las tres son muy similares por lo que por simplicidad calcularemos una sola $g(Z)$, a partir de los datos de una versión en blanco y negro de las imágenes. Haced un bucle para leer las 9 imágenes, convertirlas a BW usando `rgb2gray()` y luego pasarlas a `double` (valores entre 0 y 255). Guardad estos resultados en una matriz 3D de dimensiones `alto x ancho x 9` (n° de tomas) llamada `hdr_data`. [Adjuntad código usado](#).

Llamad a la función `muestra_HDR(hdr_data,T)` en la que podéis pinchar en distintos puntos de la imagen y ver las curvas obtenidas al graficar el valor de los píxeles en las 9 tomas. Podéis pinchar en cualquiera de las 9 imágenes (usad el botón derecho para salir). [Adjuntad una captura de la figura](#) tras pinchar en 10 puntos **variados** (claros, medios, oscuros,...). Con la información de las gráficas de la derecha es con la que trabajará nuestro algoritmo para obtener los valores de la función $g(Z)$.

Extracción de la información de un subconjunto de píxeles en las P tomas

Para no trabajar con las imágenes al completo usaremos un subconjunto de puntos, correspondiente a un submuestreo de las nueve imágenes cada n píxeles en las dos dimensiones. Escribid una función `function Zdata=sample_hdr(hdr_data,n)` para extraer esta información. Dentro de la función:

- 1) Sub-muestrear el conjunto de datos `hdr_data` quedándoos con 1 muestra por cada n en la 1ª y 2ª dimensiones (el alto y el ancho de nuestras imágenes), conservando todos los datos de la 3ª dimensión (n° de tomas). Al terminar tendréis una matriz de tamaño $N \times M \times P$. Por ejemplo, usando un salto $n=16$ el resultado sería de un tamaño $48 \times 64 \times 9$.
- 2) Luego, usad `Zdata=reshape(Zdata,N*M,P)` para reordenar estos datos de forma que en cada fila de `Zdata` tengamos los datos de cada uno de los píxeles de la malla escogida en las 9 tomas.
- 3) Finalmente sumad 1 a `Zdata`. De esta forma pasamos de los valores de píxeles $(0-255)$ a valores entre 1 y 256 que podemos usar directamente para indexar los valores de $g(Z) = (g_0, g_1, \dots, g_{255})$.

Adjuntad el código de vuestra función. Ejecutarla para un salto $n=16$ y de los datos obtenidos haced una gráfica de los valores en las 9 tomas para los píxeles nº 10, 140, 1700 y 1565 de la lista que os devuelve vuestra función. Adjuntad la gráfica resultante. ¿Cuál de los píxeles anteriores corresponde a un píxel situado en una zona muy oscura de la imagen? ¿Cuál de los píxeles creéis que proporciona más información? ¿En qué píxel se obtienen unos datos que parecen ser erróneos?

Para evitar este tipo de datos erróneos en las etapas posteriores del algoritmo los eliminaremos antes de que la función devuelva los datos. Para ello, recorrer todos los puntos (filas de Zdata) y quedarnos solo con aquellos puntos cuyos valores $\{v_k\}$ en las sucesivas exposiciones cumplan las dos condiciones siguientes:

- $(v_{k+1} - v_k) \geq 0$ para $k=1:P-1 \Rightarrow$ asegura valores crecientes en los $\{v_k\}$.
- $|v_k - (v_{k+1} + v_{k-1})/2| \leq 25$ para $k=2:P-1 \Rightarrow$ asegura que los valores de los píxeles en una exposición dada v_k no se apartan demasiado de la media de sus valores en la exposición anterior v_{k-1} y la siguiente v_{k+1} .

Al final de la función volcad el nº de puntos iniciales (tras el submuestreo) y el nº de puntos que superan las condiciones anteriores. Adjuntad el código de la función incorporando el filtrado de datos. Ejecutadla para $n=8$ y $n=16$ e indicad el nº de puntos de partida y los que superan las condiciones. Verificar que el tamaño de los datos devueltos en Zdata corresponde al nº de puntos que superan las condiciones.

2) Resolución de las ecuaciones para obtener la función $g(Z)$

Nuestro objetivo ahora, usando los datos (ya filtrados) de Zdata, es determinar la función $g(Z)$, la relación entre el valor del píxel Z en una foto y el logaritmo de la exposición E recibida en dicho píxel en esa toma: **$\log_2(E) = g(Z)$**

Para hallar los valores de g hay que resolver un sistema lineal $H \cdot g \sim b$ de carácter sobre-determinado (ajuste), con (muchas) más ecuaciones que incógnitas. El ancho de H es el número de incógnitas (los 256 valores de $g(Z)$). El número de filas de H (y de b) corresponde al número de ecuaciones Neq. Como cada punto nos da (P-1) ecuaciones hay $N \cdot (P-1)$ ecuaciones que provienen de los datos de la imagen. Adicionalmente se impondrán otras 254 ecuaciones adicionales, $g_k = (g_{k-1} + g_{k+1})/2$, (para $k=2, \dots, 255$) tratando de forzar que los valores $g_0, g_1, \dots, g_{255}$ sean crecientes.

Como típicamente N es del orden de miles de puntos (N) y tenemos $P=9$ tomas el número de ecuaciones será del orden de decenas de miles. Afortunadamente en cada ecuación solo aparecerán 2 o 3 incógnitas, por lo que los valores de la matriz H serán casi todos nulos. En estos casos es un desperdicio reservar una matriz con millones de "casillas" cuando más del 99% de sus entradas son cero. Para evitarlo usaremos matrices de tipo "sparse" (matrices poco pobladas o casi vacías). En este tipo de matrices solo se guardan aquellos elementos no nulos de la matriz, y se asume que cualquier elemento no explícitamente especificado es cero.

Por ejemplo la matriz $H = \begin{pmatrix} 1 & 0 & 0 & 3 \\ 0 & 0 & 0 & -1 \\ 0 & 2 & 0 & 0 \end{pmatrix}$ se crearía $H = [1 \ 0 \ 0 \ 3; \ 0 \ 0 \ 0 \ -1; \ 0 \ 2 \ 0 \ 0];$

Usando matrices "sparse" haríamos: $i = [1 \ 1 \ 2 \ 3]; j = [1 \ 4 \ 4 \ 2]; v = [1 \ 3 \ -1 \ 2];$
 $H2 = \text{sparse}(i,j,v,3,4)$

Al hacer $(H2-H)$ nos dará una matriz de 0's, ya que H y H2 son la misma matriz.

Para estas matrices "sparse" creamos previamente dos vectores con la colección de coordenadas (filas i y columnas j) de los elementos no nulos y otro vector (mismo tamaño) v con los valores de esas casillas. Luego la matriz se crea con la función `sparse()` pasándole estos vectores i,j,v y el tamaño (3x4) deseado para la matriz.

Nuestro problema es ideal para usar este tipo de matrices, ya que en cada fila de la matriz H solo hay un par de coeficientes no nulos. Se trata de escribir una función `function g = solve_G(Zdata,T)` que reciba la matriz Zdata (PxN con los valores de N puntos en las P tomas) y el vector T con los P tiempos de exposición. Dentro de la función crearemos la matriz "sparse" H y el vector b, resolviendo el sistema $H \cdot g \sim b$, cuya solución es un vector (256x1) con los valores de la función g(Z) en los 256 posibles valores de los píxeles. Este será el vector devuelto por la función. Veamos primero los pasos para crear la matriz H y el vector b del sistema:

- 1) Determinar el n° de ecuaciones Neq a partir del n° de píxeles usados N y el n° de tomas P (las dimensiones de Zdata). Recordad las 254 ecuaciones "extra". Cread un vector b (inicializado con 0's) de dimensiones (Neq x 1). Este vector b no será de tipo "sparse" ya que la mayoría de sus entradas no son nulas.
- 2) Para la matriz "sparse" H hay que rellenar previamente los vectores i, j con las coordenadas (i=fila, j=columna) de las entradas no nulas y un tercer vector v con sus valores. En este caso podemos determinar el número de elementos no nulos (la dimensión de los vectores i, j y v) sabiendo que en cada ecuación procedente de los datos de las tomas solo aparecen dos entradas no nulas. Por el contrario, las 254 ecuaciones impuestas para "regularizar" g(Z) tienen tres elementos no nulos (en ellas aparecen g_{k-1} , g_k y g_{k+1}). Reservar espacio para los correspondientes vectores i, j y v. **Para los datos obtenidos usando n=8, indicad el n° de ecuaciones del sistema y el número de entradas no nulas en la matriz H. ¿Qué % suponen del total de la matriz?.**
- 3) Para las primeras ecuaciones barred todos los píxeles desde 1 hasta N. En cada uno de ellos extraed los P valores {Z} de las tomas y de entre ellos hallad el más cercano a 128 (Zref). Restando la ecuación con dicho valor a las demás, se obtienen las siguientes (P-1) ecuaciones:

$$\begin{aligned} g(Z_1) - g(Z_{ref}) &= \log_2(T_1 / T_{ref}) \\ g(Z_2) - g(Z_{ref}) &= \log_2(T_2 / T_{ref}) \\ &\dots \\ g(Z_M) - g(Z_{ref}) &= \log_2(T_M / T_{ref}) \end{aligned}$$

Veamos cómo ir añadiendo estas ecuaciones al sistema. Cada una de estas ecuaciones involucra únicamente a 2 incógnitas, por lo que añadirá 2 entradas a los vectores i , j y v . En los dos casos el valor de la i es el mismo: el n° de ecuación en la que estamos (llevar un "contador" de ecuaciones). Los valores de las columnas j serán las incógnitas involucradas en la ecuación, los valores de Z_k y Z_{ref} . En cuanto a los valores de v (los contenidos de la matriz H) serán 1 y -1, los coeficientes que acompañan a g_k y g_{ref} en estas ecuaciones. En el caso de b basta guardar el lado derecho de la ecuación, $\log_2(T_k/T_{ref})$, en la fila de b correspondiente al n° de ecuación en la que nos encontremos. Tras rellenar las dos entradas de i , j y v , y la correspondiente componente de b , avanzaremos el contador del n° de ecuación y procederemos a añadir los datos de la siguiente.

Tras añadir las $(P-1)$ ecuaciones procedentes de un píxel extraemos los datos del siguiente (siguiente fila de Z_{data}) y repetimos el proceso. Al terminar con los N píxeles el contador de ecuaciones debe haber llegado al valor $N \cdot (P-1)$.

- 4) Tras terminar con las ecuaciones de los N datos de la imagen, debemos añadir las ecuaciones, $g_k = (g_{k-1} + g_{k+1})/2$, que tratan de asegurar que los valores de g sean crecientes. Dichas ecuaciones son equivalentes a:

$$\begin{aligned} -g_0 + 2g_1 - g_2 &\approx 0 \\ -g_1 + 2g_2 - g_3 &\approx 0 \\ &\dots \end{aligned}$$

En este caso cada una de las ecuaciones añade 3 valores a i , j y v , al involucrar a tres incógnitas. Los valores de i son como antes el n° de ecuación en la que nos encontramos. Los valores de j son los índices de las incógnitas que aparecen en la ecuación: (1,2,3) para la 1ª ecuación, (2,3,4) para la 2ª, y así sucesivamente hasta llegar a (254,255,256) para la ecuación n° 254. Finalmente, los valores de v añadidos son siempre -1,2,-1, los coeficientes que acompañan a las incógnitas. En cuanto al vector b , no hará falta modificarlo ya que todas estas ecuaciones están igualadas a 0 (y el vector b se creó lleno de 0's).

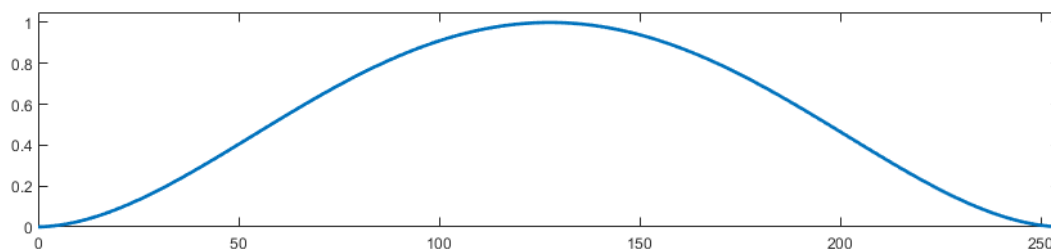
- 5) Una vez que hemos rellenado los vectores i , j y v , creamos la matriz H "sparse" del sistema haciendo $H = \text{sparse}(i, j, v, \text{Neq}, 256)$ indicándole que nuestro sistema tiene Neq ecuaciones (filas) y 256 incógnitas (columnas).

Al terminar haced `whos H` para comprobar que sus dimensiones son las correctas. Una vez creada, podremos hacer con H casi todas las operaciones que haríamos con una matriz "normal" en Matlab. Por ejemplo, con `sum(H(:,1)~=0)`, podéis obtener el n° de entradas no nulas en 1ª columna de H , (debe daros 1922).

Generad la matriz H para los puntos generados con $n=8$ y [adjuntad el plot del vector obtenido con `sum\(H~=0\)`](#). ¿Qué representa la gráfica obtenida? Indicad el número de veces que aparecen las incógnitas g_{10} y g_{20} en las ecuaciones.

Haced hasta aquí para la próxima clase.

Uso de pesos: en la práctica se suelen ponderar estas ecuaciones en función de su importancia o fiabilidad. Esto se hace al procesar cada ecuación, multiplicando tanto los valores v de la matriz H como los del vector b por el peso W que decidamos para esa ecuación. Para fijar estos pesos definiremos un vector de pesos w con un valor máximo cerca del centro (128) y bajando hasta casi 0 para los valores de los píxeles cerca de 0 o 255 (cuya información es menos fiable, ya que es posible que estén saturados). Una posible elección para estos pesos (ilustrada en la figura adjunta) se puede obtener con: $M = 256$; $t=(1:M)'/(M+1)$; $w=(t.*(1-t)).^2$; $w=w/\max(w)$;



Usando este vector w , a cada ecuación procedente de los datos de las imágenes le asignaremos un peso $W=\sqrt{w(Z_k) \cdot w(Z_{ref})}$, la media geométrica de la "fiabilidad" de los dos valores (Z_k y Z_{ref}), que aparecen en la ecuación. Recordad que los valores de las Z 's pueden usarse directamente como índices de la tabla $w()$ porque se les sumó 1 al extraerlos (para que tuvieran valores de 1 a 256 en vez de 0 a 255). El peso W calculado multiplica tanto a los valores v como al vector b para esa ecuación. **Indicad el peso que usaríais para la ecuación $g_{50} - g_{12} = \dots$**

Como las 254 ecuaciones añadidas al final son "teóricas" (no se basan en los datos extraídos de las imágenes) no es necesario aplicarles este tipo de pesos asociados a la fiabilidad de los datos. Lo que si haremos es multiplicar estas ecuaciones por un peso λ constante con el que podemos regular la importancia que queremos dar a las ecuaciones teóricas frente a las procedentes de datos de las exposiciones. Si p.e se usa $\lambda=1$, estamos dando igual importancia a ambos tipos de ecuaciones. Podéis incorporar λ como un parámetro más en `solve_G(Zdata,T,lambda)` para poder modificarlo fácilmente. **Adjuntad el código para crear H y b incorporando pesos.**

Resolución del sistema para obtener $g(Z)$:

Tras construir H y b el ajuste $H \cdot g \sim b$ basta resolverlo en Matlab como $g=H \backslash b$ (igual que con matrices "normales"). El problema es que, como vimos en clase, la solución de este problema no es única: si se desplaza el vector solución g , el resultado será también una solución del problema. Matlab os avisará de esto al resolver dando un warning indicando que el rango de la matriz H (ancho 256) es sólo de 255.

Una convención es hacer que el valor de g_{128} , correspondiente a la componente $g(129)$, sea 0. Para ello, una vez que Matlab ha resuelto el sistema, basta restar a g el valor de $g(129)$. **¿Qué valor obtenéis para g_{128} antes de hacer esta corrección (usando los datos submuestreados con $n=8$ y $\lambda=1$)?**

Con esto la función `solve_G()` ya está completa ([adjuntad su código](#)) y la podremos usar en un script donde juntaremos todos los pasos descritos hasta ahora:

- Leer las imagenes y crear `hdr_data` (alto x ancho x P) + vector T de tiempos.
- Definir el salto n y usar `extraer_datos()` para obtener la información de los N píxeles seleccionados en las P tomas en la matriz Zdata (N x P).
- Usar `solve_G()` para hallar los valores de g, un vector columna de tamaño 256 con los resultados de ($g_0, g_1, \dots, g_{255}$).

Usando $n=8$ y $\lambda=1$, resolver y [adjuntad una gráfica de la función \$g\(z\)\$ en función del valor de píxel \(de 0 a 255\)](#). ¿Es la función $g(z)$ obtenida creciente? Volcad con 4 decimales los 5 primeros valores del vector g solución.

Adjuntad ahora la gráfica de $g(Z)$ obtenida con $n=8$ y usando $\lambda=0$ (lo que hace que las 254 ecuaciones de regularización no tengan en cuenta). Veréis que el resultado es un poco más irregular pero aceptable, ya que al tener bastantes datos el uso de las ecuaciones de "regularización" no es crítico. Por el contrario si se dispone de pocos datos, el resultado puede ser muy incorrecto si no usamos las ecuaciones de regularización. [Adjuntad la gráfica obtenida para \$g\(Z\)\$ con \$n=64\$ y \$\lambda=0\$. Indicad el número de puntos N devueltos en Zdata y el nº de ecuaciones planteadas. Justificad el aspecto de esta curva](#). Puede ser ilustrativo ver la gráfica que hicimos antes a partir de la matriz H con las veces que aparece cada valor $g(Z)$ en las ecuaciones para $n=64$. [Repetir la gráfica para \$n=64\$ pero usando ahora \$\lambda=5\$](#) .

3) Estimar la radiancia R de la escena

En este apartado usaremos siempre para $g(Z)$ la solución obtenido usando los datos Zdata extraídos con un sub-muestreo $n=8$ y $\lambda=1$ al construir H.

Conocidos estos valores de $g(Z)$, podemos obtener la exposición E que reciben los píxeles en una toma sabiendo que $\log_2(E)=g(Z)$. Una vez conocida E y sabiendo que $E=R \cdot T$, siendo T el tiempo de exposición de esa toma) podemos estimar R (radiancia de la escena). Para cada píxel, usando sus valores en las P tomas $\{Z_1, Z_2, \dots, Z_P\}$ y los tiempos de exposición de cada toma $\{T_1, T_2, \dots, T_P\}$ el logaritmo ($\log_2$) de la radiancia R de la escena puede obtenerse con las siguientes ecuaciones:

$$\begin{aligned} \log_2 R &\approx g_{z_1} - \log_2(T_1) \\ \log_2 R &\approx g_{z_2} - \log_2(T_2) \\ &\dots \\ \log_2 R &\approx g_{z_m} - \log(T_M) \end{aligned}$$

Al ser la radiancia R de la escena común en todas las tomas podríamos estimarla con cualquiera de esas ecuaciones a partir de los datos de una única toma.

Usad los datos de la 1ª exposición ($T=1/1000$) guardada en `hdr_data(:, :, 1)` junto con la primera de las ecuaciones anteriores para estimar el $\log_2$ de la radiancia de la escena. [Adjuntad el código usado \(puede hacerse con una sola línea de MATLAB\).](#)

Para visualizar la imagen $\log_2 R$ obtenida usaremos una técnica de pseudo-color, asociando un color distinto a los píxeles de la imagen en función del valor de $\log_2 R$, como si se fuese un mapa de "temperaturas". Visualizar la imagen obtenida con `imagesc()` y usad "hot" como paleta de color (función `colormap`). Superponer en la figura una barra de color (función `colorbar`) para ilustrar el rango de valores de $\log_2(R)$. [Adjuntad la imagen resultante.](#) Al trabajar con el logaritmo en base 2, el rango de valores mostrado corresponde al rango de "stops" fotográficos que cubre la escena. [Indicad el rango dinámico de la escena en stops.](#)

El problema de usar una sola toma para estimar $\log_2(R)$ son los píxeles con valores cercanos a 0 o 255. En estos casos, al estar cerca de la saturación, la estimación de $\log_2(E)$ a partir de $g(Z)$ no es fiable, lo que da lugar a un nivel alto de ruido en la estimación de $\log_2(R)$, que es claramente visible en la imagen anterior.

Para evitar este problema, para obtener la radiancia definitiva promediaremos los valores de $\log_2(R)$ procedentes de los datos de las 9 exposiciones. Además, al hacer esta media, daremos más importancia a los resultados procedentes de valores de Z cercanos a 128 (más alejados de 0 o 255 y por lo tanto más fiables), usando para ello el mismo vector de pesos w definido antes. Todo esto se implementará en una función `function log2R = get_log2R(hdr_data, g, T)`

La función recibe los datos de exposiciones de las 9 tomas (`hdr_data`), el vector g (256×1) con los valores de $g(Z)$ que hemos estimado (con $n=8$, $\lambda=1$) y el vector de tiempos T . A partir de estos datos la función devolverá una imagen (ancho x alto) con el logaritmo (en base 2) de la radiancia de la escena. El proceso detallado sería:

1. Reservar una matriz $\log_2 R$ (alto x ancho) para guardar la imagen final.
2. Barrer todos los píxeles (alto x ancho) y para cada uno de ellos:
 - Extraer los datos $\{Z_1, Z_2, \dots, Z_P\}$ de ese píxel en las P tomas.
 - Con la información del vector g y del vector de tiempos T , calcular los P posibles valores de la radiancia, $V_k = g(Z_k) - \log_2(T_k)$, para cada toma.
 - Calcular los pesos $W = \{W_1, W_2, \dots, W_P\}$ correspondientes a $\{Z_1, Z_2, \dots, Z_P\}$ usando el vector w de pesos. Dividir el vector W obtenido por la suma de sus valores de forma que ahora la suma de sus valores sea 1.
 - Hacer la media ponderada de los P valores $\{V_k\}$ de la radiancia usando los pesos W . Guardad el resultado, nuestra estimación final de $\log_2(R)$ para ese píxel, en la correspondiente posición de la matriz $\log_2 R$. La media ponderada de P valores $\{V_k\}$ con pesos $\{W_k\}$ puede calcularse como el producto escalar:

$$\sum_{k=1}^P V_k \cdot W_k$$

Para el producto escalar en matlab podéis usar la función `dot()`. [Adjuntad vuestro código de `get\_log2R\(\)`.](#)

Añadid `get_log2R()` al script anterior para completar el proceso y ejecutadlo usando como antes un submuestreo de $n=8$ y un peso $\lambda=1$. [Adjuntad la imagen resultante](#). El resultado debe tener menos ruido que la obtenida antes usando una sola toma. [¿Cuál es ahora el rango dinámico de la escena expresado en stops?](#) [¿Es menor o mayor que antes? Justificar.](#)

Visualización de la imagen HDR en color

La imagen `log2R` calculada en el apartado anterior corresponde a una versión en blanco y negro de la escena, ya que se construyó usando los datos de `hdr_data`, donde previamente habíamos guardado la luminancia de las exposiciones.

En una imagen en color por cada píxel hay que calcular 3 valores de `log2R` (uno por cada canal), cada uno obtenido a partir de los valores de los píxeles de ese canal en las 9 tomas. La forma más sencilla de hacerlo con las funciones disponibles es volver a cargar las 9 imágenes "`belg_X.jpg`" guardando ahora en `hdr_data` los datos del canal rojo de cada toma (en vez de su versión en grises). Si llamamos ahora a `get_log2R(hdr_data,g,T)` tendremos el valor de `log2R` correspondiente al canal rojo. Notad que los valores de $g(Z)$ no se recalculan, usándose la misma tabla $g(Z)$ que se calculó con los datos de la luminancia. Repitiendo el proceso para el canal verde y el azul obtendríamos los 3 planos de color de `log2R`, que podemos juntar para componer una imagen en color.

Una vez estimado el logaritmo de la radiancia `log2R` para los 3 planos, basta hacer `im_hdr=2.^log2R` para obtener la radiancia de la escena, esto es, la imagen HDR. El rango dinámico de una escena es el cociente entre los valores máximo y mínimo de la radiancia. [Indicad máximo/mínimo de la radiancia obtenida y su rango dinámico.](#)

Podéis visualizar `im_hdr` con `imshow(im_hdr/M)`, donde `M` es el máximo de `im_hdr` calculado antes, para que la imagen que recibe `imshow()` tenga valores entre 0 y 1. [Adjuntad la imagen resultante. ¿Cuál es el problema?](#)

Para visualizar estas imágenes HDR con las limitaciones de la pantalla tendremos que aplicarle un algoritmo adicional de "tonemapping". Este tipo de algoritmos tratan de ajustar el rango de valores de la imagen HDR al (mucho más limitado) rango de la pantalla. Para conseguir esto se suelen usar técnicas de ajuste local de contraste, para preservar el detalle tanto en las zonas claras como oscuras.

Una función de MATLAB que aplica este tipo de algoritmos es `tonemap()`: recibe una imagen HDR (tipo float) y devuelve una imagen uint8 (con un rango entre 0 y 255) lista para ser visualizada. Además podemos añadirle parámetros opcionales para cambiar la saturación y otras características de la imagen final. Probadla con: `im_final=tonemap(im_hdr,'AdjustSaturation',3);` [Visualizad la imagen final tras pasar por el "tonemapping" usando imshow\(\) y adjuntad una captura.](#)

4) HDR usando la pirámide laplaciana

El algoritmo usado en los apartados anteriores se basaba en plantear la relación existente entre la radiancia de la escena y la exposiciones capturadas por el sensor, conocidos sus tiempos de exposición. Su ventaja es que puede recuperar (salvo un factor) información sobre la radiancia real de la escena. Sin embargo, si el objetivo es simplemente mezclar fotos con distinta exposición, combinando en una imagen las partes que mejor se ven de unas u otras, hay métodos más sencillos.

En este apartado vamos a implementar uno de esos métodos basado en la pirámide laplaciana, implementada en un proyecto anterior. El método consiste en hallar la pirámide laplaciana de las imágenes de partida para luego combinarlas (siguiendo ciertas reglas) en una sola pirámide. Invirtiendo la pirámide compuesta se obtiene la imagen HDR final.

En este caso partiremos de tres imágenes (exp_1, exp_2 y exp_3) con diferentes exposiciones (aquí no se necesita conocer sus tiempos de exposición). El algoritmo tal como se describe opera con imágenes de tipo double con un **rango entre 0 y 1**.

Pirámide Laplaciana: usad la función `lap_pir()` que escribisteis en un proyecto anterior para obtener la pirámide laplaciana de las 3 imágenes (con $N=5$ niveles).

Combinación de las pirámides laplacianas: combinad las tres pirámides en una única pirámide siguiendo estas reglas:

- Para el nivel N de la pirámide (versión reducida de las imágenes) haced la media de ese nivel en las 3 pirámides que tenemos.
- Los otros niveles ($1:N-1$) guardan el detalle de las imágenes. La idea de este algoritmo es que las zonas muy oscuras (por falta de exposición) o demasiado claras (exposición excesiva) van a tener menos detalle que las correctamente expuestas. Se trata de conservar en cada escala la información de aquella imagen con el máximo detalle (valores más altos en el correspondiente nivel de la pirámide laplaciana). Como nuestras imágenes son en color, para decidir qué píxel tiene el mayor detalle usaremos la suma (en valor absoluto) de las componentes r, g, b del detalle.

Barrer los niveles de 1 a $N-1$ y dentro de cada nivel barrer todos los píxeles. Para cada píxel determinar cuál de las 3 pirámides tiene un mayor detalle (comparando el valor de $|r|+|g|+|b|$ de las 3 imágenes para ese nivel). Una vez determinado el píxel con más detalle, usad el vector `rgb` "ganador" para rellenar el correspondiente píxel de ese nivel en la pirámide combinada.

[Adjuntar código usado para combinar las 3 pirámides](#)

Inversión de la pirámide mezcla: tras obtener la pirámide combinada solo queda "colapsarla" de vuelta (usando la función `inv_lap`) para obtener la imagen HDR.

Retoques finales: un problema de la imagen HDR así obtenida es que no está garantizado que sus valores se mantengan entre 0 y 1 (al proceder de una pirámide laplaciana mezcla de varias). Por lo tanto, lo primero es calcular sus valores mínimo y máximo y usarlos para re-escalar la imagen final entre 0 y 1.

La imagen final combina el detalle de las diferentes imágenes, tanto en las zonas de sombra como en los claros. El problema es que, al igual que vimos en el apartado anterior, este detalle puede perderse de nuevo al tratar de visualizar la escena. Una forma de evitarlo es hacer una ecualización local del histograma. En el Tema 4 vimos cómo una ecualización del histograma aumenta el contraste de una imagen. El hecho de ser local hace que funcione tanto en las zonas claras como oscuras. En MATLAB podemos usar la función:

```
I=adapthisteq(I, 'ClipLimit', clip);
```

que implementa el algoritmo sobre una imagen 2D. El parámetro clip es la fracción de píxeles saturados (por debajo de 0 o por encima de 1) admitidos en la imagen final. Como vimos en el proyecto de revelado de una imagen "raw" cuanto más alto sea dicho porcentaje más "dramático" (y menos realista) será el resultado.

Un valor adecuado puede ser 0.01.

Esta ecualización se aplicará solo a la luminancia (versión B/W de la imagen). Para ello, convertir la imagen obtenida (una vez re-escalada entre 0 y 1) al espacio HSV (Hue, Saturación, Value) y aplicad la función **adapthisteq** al canal de luminancia V, usando un valor de 0.01 para el parámetro **'ClipLimit'**.

Ya que estamos en el espacio HSV aumentaremos también la saturación del color, aplicando la función $T(x)=x^{0.7}$ al canal de saturación S. [Adjuntad el código usado para re-escalar la imagen, pasar al espacio HSV, aplicar la ecualización adaptativa a la luminancia V y el aumento de saturación al canal S.](#)

Regresad al espacio RGB para visualizar la imagen modificada y [adjuntad captura del resultado.](#)